

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

EXPERIMENT 9

Student Name: Aryan Yadav

UID: 24BCF10079

Branch: AIT-CSE (FSD)

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: Computer Network

Subject Code: 24CSH-337

Experiment

Implementing a Connectionless Communication Using UDP

Aim

To build a UDP client–server application using Python socket programming and observe how a connectionless session behaves in practice.

Objective

- Understand the working principles of UDP and its connectionless model of communication.
 - Implement a UDP client–server connection using Python's socket module.
 - Establish communication between a server.py and a client.py script over UDP sockets.
 - Observe how datagrams are exchanged without any connection setup or teardown during a live UDP session.
-

Theory

UDP (User Datagram Protocol) is a connectionless, transport-layer protocol that sends data as independent datagrams without any prior setup between the two hosts. It follows a client–server model, but unlike TCP, the server does not listen for or accept connections: it simply binds a socket to an IP address and port and waits for datagrams to arrive, while the client sends data straight to that address using its own socket.

Because there is no session to maintain, UDP offers no guarantee that a datagram is delivered, delivered once, or delivered in order. There is also no three-way handshake before data flows — the very first packet sent already carries data, which removes the setup delay that TCP incurs. This trade-off of reliability for speed is what makes UDP the protocol of choice for latency-sensitive applications such as DNS lookups, live streaming, VoIP calls, and online gaming, where a late or duplicate packet is less costly than a delayed one.

In this experiment, the server calls `bind()` to attach itself to an address and port, then loops on `recvfrom()`, which returns both the incoming data and the address of whoever sent it — this is necessary because, without a connection, the server has no other way of knowing who to reply to. The client calls `connect()` purely as a local convenience so that it can use `send()` and `recv()` instead of repeating the server's address on every call; no actual connection is established on the network, since UDP has no concept of one.

Practical Steps

1. Open the project folder in a code editor such as VS Code, PyCharm, or IntelliJ.
2. Create two Python files named `server.py` and `client.py`.
3. Paste the server-side code into `server.py` and the client-side code into `client.py`.
4. Save both files.
5. Open two terminals (or a split terminal).
6. In Terminal 1, start the server with `python server.py`.
7. In Terminal 2, start the client with `python client.py`.
8. Exchange a few messages between the client and server CLIs, then type `Over` on the client side to end the session.

EXPERIMENT 9

Code

server.py

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind(("127.0.0.1", 5000)) # bind the server to an IP and port

print("UDP Server started")
print("Waiting for messages...")

while True:
    message, client_address = server_socket.recvfrom(1024)
    message = message.decode()
    print("Client:", message)
    if message == "Over":
        break
    reply = input("Enter reply: ")
    server_socket.sendto(reply.encode(), client_address)

server_socket.close()
print("Server closed")
```

client.py

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.connect(("127.0.0.1", 5000))
print("UDP Client started")

while True:
    message = input("Enter message: ")
    client_socket.send(message.encode()) # encode(): string -> bytes
    if message == "Over":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)

client_socket.close()
print("Client closed")
```

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

EXPERIMENT 9

Result

server.py — Terminal 1

```
PS D:\Aryan\CN_Lab\UDP_Experiment> python server.py
UDP Server started
Waiting for messages...
Client: Hey, this is Aryan (24BCF10079)
Enter reply: Hi Aryan, got your datagram fine
Client: Sending a short test message now
Enter reply: Received it, no handshake needed on this end
Client: Good, I'll wrap up the session
Enter reply: Sounds good, closing up
Client: Over
Server closed
```

client.py — Terminal 2

```
PS D:\Aryan\CN_Lab\UDP_Experiment> python client.py
UDP Client started
Enter message: Hey, this is Aryan (24BCF10079)
Server: Hi Aryan, got your datagram fine
Enter message: Sending a short test message now
Server: Received it, no handshake needed on this end
Enter message: Good, I'll wrap up the session
Server: Sounds good, closing up
Enter message: Over
Client closed
```

Learning Outcome

- Understood the working and characteristics of UDP, including its connectionless nature and best-effort delivery.
- Built a working UDP client–server application using Python socket programming.
- Analyzed how datagrams are exchanged between a client and a server without any connection setup or teardown.